

CONTENTS

Práctico 7 (Ensamblador 8086) — Arquitectura de Computadoras

FING UdelaR · Resolución completa y verificada

Preguntas teóricas

Ejercicio 1 ★★ — Suma con detección de overflow

Ejercicio 2 ★★ — Copia de N bytes (memcpy)

Ejercicio 3 ★★ — Comparación de N bytes

Ejercicio 4 ★★ — Búsqueda en tabla de bytes

Ejercicio 5 ★★ — Suma de vector con detección de posición del carry

Ejercicio 6 ★★ — Eliminar espacios de un string

Ejercicio 7 ★ — Layout de struct en memoria

Ejercicio 8 ★★★ — Substring (buscar un string dentro de un texto)

Ejercicio 9 ★★★ — Peso de un arreglo de matrices 4×4

Ejercicio 10 ★★★ — Protocolo de pasaje de parámetros por pila

Resumen de técnicas usadas en este práctico

Práctico 7 (Ensamblador 8086) — Arquitectura de Computadoras

FING UDELAR · RESOLUCIÓN COMPLETA Y VERIFICADA

⚠ **Nota general sobre el material de referencia:** el material oficial de la cátedra ("Ejercicios Resueltos.pdf") solo trae resueltas las preguntas teóricas y los ejercicios 1 a 5 y 7. Los ejercicios 6, 8, 9 y 10 **no tienen solución oficial** — se resolvieron acá desde cero, con la misma metodología (alto nivel → assembler, protocolo de registros explícito, verificación instrucción por instrucción).

Además, revisando instrucción por instrucción el código oficial de los ejercicios 1 a 5, se encontraron **tres errores reales** (no solo de estilo) en el material del curso, documentados y corregidos en el lugar correspondiente: un registro perdido en el Ejercicio 1, una copia de palabras en vez de bytes en el Ejercicio 2, y un flag pisado por una instrucción intermedia en el Ejercicio 3 (este último es el más grave: rompe la lógica de comparación). También hay un error conceptual leve en la pregunta teórica (g).

Convención usada en toda la resolución: números hexadecimales con sufijo `h` o prefijo `0x` indistintamente. Segmento por defecto según la letra: variables en `DS` salvo que se indique lo contrario (así lo aclara la letra del práctico). `BX`, `BP`, `SI`, `DI` son los únicos registros que pueden formar parte de un operando de memoria en 8086 (direccionamiento base+índice: solo las combinaciones `BX+SI`, `BX+DI`, `BP+SI`, `BP+DI` son válidas; `AX`, `CX`, `DX` nunca pueden aparecer dentro de corchetes).

Preguntas teóricas

(a) La siguiente instrucción a ejecutar está determinada por el par `CS:IP`: `CS` (Code Segment) fija el segmento de código en ejecución, e `IP` (Instruction Pointer) el desplazamiento dentro de ese segmento. La dirección física se calcula como siempre: $CS \times 16 + IP$. Al terminar de ejecutar cada instrucción, la CPU incrementa automáticamente `IP` en la longitud (en bytes) de la instrucción recién ejecutada, de modo que vuelve a apuntar a la próxima instrucción — salvo que la instrucción ejecutada haya sido un salto/ `CALL` / `RET` / interrupción, en cuyo caso `IP` (y eventualmente `CS`, si es un salto *far*) se sobrescribe directamente con el destino.

(b) El 8086 es **CISC** (Complex Instruction Set Computer): tiene instrucciones de longitud variable (1 a 6 bytes), modos de direccionamiento complejos que combinan registros e desplazamientos en una sola instrucción (`[BX+SI+4]`), instrucciones que hacen mucho trabajo internamente (`MUL`, `DIV`, las instrucciones de string con prefijo `REP`), y un conjunto grande y heterogéneo de instrucciones — todo lo opuesto a la filosofía RISC (instrucciones simples, de longitud fija, que hacen poco cada una).

(c) Una **dirección segmentada** es un par `SEGMENTO:DESPLAZAMIENTO` que no es la dirección real de memoria, sino una fórmula para calcularla. El 8086 la transforma en una dirección física de 20 bits mediante:

| Dirección física = $SEGMENTO \times 16 + DESPLAZAMIENTO$

(equivalente a agregar un `0` hexadecimal a la derecha del segmento — desplazarlo 4 bits — y sumarle el desplazamiento). Esto permite direccionar 1 MB (20 bits) con registros de 16 bits.

(d) Como el segmento se multiplica por 16, **existen múltiples pares** `SEGMENTO:DESPLAZAMIENTO` **distintos que producen la misma dirección física** (hay 4096 formas de expresar cada dirección física mediante distintos pares). Si dos segmentos se solapan en el rango de direcciones físicas que cubren, escribir a través de uno puede modificar sin darse cuenta datos que “pertenecen” al otro segmento — en el fondo es la misma memoria física. El cuidado a tener es, entonces, **organizar los segmentos de manera que no se solapen** (o, si se solapan intencionalmente, ser consciente de qué otro segmento comparte esa región antes de escribir).

(e) `PUSH src` : decrementa `SP` en 2 y luego escribe `src` (16 bits) en la posición de memoria `SS:[SP]` (el nuevo tope de la pila). `POP dest` : lee los 16 bits en `SS:[SP]` (el tope actual) hacia `dest`, y luego incrementa `SP` en 2. Los registros implícitamente relacionados son `SP` (se modifica en ambas) y `SS` (segmento donde vive la pila, usado implícitamente sin necesidad de indicarlo). La pila crece hacia direcciones **bajas**: cada `PUSH` resta a `SP`, cada `POP` le suma.

(f) Dos ejemplos claros de instrucciones con registros implícitos, más allá de `PUSH / POP` :

- `MUL / IMUL / DIV / IDIV` : usan implícitamente el par `DX:AX` (o solo `AX` en la versión de 8 bits) como uno de los operandos y como destino del resultado — nunca se especifican en la instrucción.
- `LOOP etiqueta` : decrementa implícitamente `CX` y salta a `etiqueta` mientras `CX≠0`, sin que `CX` aparezca como operando explícito.

(`PUSHF / POPF`, la respuesta que trae el material oficial, también es válida — usan implícitamente el registro `FLAGS` — pero `MUL / DIV / LOOP` son ejemplos más representativos de “registro implícito” en el sentido en que lo pregunta la letra, porque son las instrucciones aritméticas/de control de flujo de uso diario, mientras que `PUSHF / POPF` casi no se usan en este curso.)

(g) Las banderas **aritméticas** (las que reflejan el resultado de una operación aritmética/lógica) son: `CF` (Carry), `PF` (Parity), `AF` (Auxiliary carry), `ZF` (Zero), `SF` (Sign) y `OF` (Overflow). Son 6, no “todas”.

⚠ **Corrección al material oficial:** la respuesta de la cátedra a (g) es simplemente “Todas”. Esto es impreciso: el registro `FLAGS` del 8086 tiene 9 bits definidos, y no todos son banderas *aritméticas*. `TF` (Trap, para debugging paso a paso), `IF` (Interrupt enable, habilita/deshabilita interrupciones enmascarables) y `DF` (Direction, controla el sentido de auto-incremento/decremento de `SI / DI` en instrucciones de string) son banderas de **control**, no reflejan el resultado de ninguna operación aritmética — no tiene sentido preguntar “¿qué banderas se activaron al sumar?” y responder “también `IF`”. La respuesta correcta y completa es: `CF, PF, AF, ZF, SF, OF` (6 banderas aritméticas de un total de 9 banderas definidas).

Ejercicio 1 ★★ — Suma con detección de overflow

Enunciado: recibe dos números en `AX` y `BX`, los suma, y si hay overflow, vuelca `AX, BX, CX, DX, SI, DI` a partir de `0x0800:0x0100`.

Alto nivel:

```

short suma(short ax, short bx) {
    short resultado = ax + bx;
    if (hubo overflow con signo) {
        volcar(0x0800:0x0100, ax_actual, bx, cx, dx, si, di);
    }
}

```

"Overflow" en una suma de números **con signo** (`short`) se detecta con la bandera `OF`, no con `CF` (que es para overflow sin signo/acarreo) — distinción que la letra no dice explícitamente pero que se desprende de que los operandos son "números" genéricos tratados como con signo por defecto en este curso, salvo que se indique `unsigned`.

```

ej1 PROC
    ADD AX, BX          ; (1) AX <- AX+BX; OF queda en 1 si hay overflow con signo
    JNO _fin            ; (2) si no hubo overflow, terminamos sin volcar nada

    PUSH CX             ; (3) usamos CX como temporal para cargar ES (lo restauramos ya mismo)
    MOV CX, 0x0800
    MOV ES, CX
    POP CX              ; (4) CX vuelve a su valor original, sin haber sido observado por nadie

    MOV ES:[0x0100], AX  ; (5) AX ya contiene la SUMA (que es la que desbordó)
    MOV ES:[0x0102], BX
    MOV ES:[0x0104], CX
    MOV ES:[0x0106], DX
    MOV ES:[0x0108], SI
    MOV ES:[0x010A], DI

_fin:
    RET
ej1 ENDP

```

Revisión línea por línea:

- (1)-(2): `ADD AX, BX` dobla como "la suma" y como el disparador del chequeo — no hace falta instrucción aparte. `JNO` (Jump if Not Overflow) es el salto correcto para `OF`, según la cartilla (ver guía temática §4.3).
- (3)-(4): para poner `0x0800` en `ES` hace falta pasarlo por un registro general primero (no existe `MOV ES, inmediato` en 8086). Se usa `CX` como intermediario porque no participa en la lista de registros a volcar de otra forma en este tramo, y se restaura antes de ser leído — por lo tanto su valor final en el volcado es el original, sin alterar.
- (5): la dirección `ES:[0x0100]` es direccionamiento **directo a memoria** (constante, sin registro), con segmento `ES` explícito — coincide con `0x0800:0x0100` de la letra. Offsets consecutivos de a 2 bytes por ser todos registros de 16 bits: `AX` (100h), `BX` (102h), `CX` (104h), `DX` (106h), `SI` (108h), `DI` (10Ah) — verificado: 6 registros × 2 bytes = 12 bytes, `0x100+0x0A=0x10A` para el último. ✓

⚠ **Error encontrado en el material oficial:** la solución de la cátedra hace `PUSH AX` antes del `ADD AX, BX`, y luego, tras detectar el overflow, hace `POP AX` **antes de volcar los registros** — lo que restaura el valor de `AX` **previo a la suma** y lo graba en memoria en el lugar de "AX". Es decir: el volcado que hace la solución oficial guarda el `AX` de **entrada** (antes de sumar), no la suma que efectivamente desbordó. Esto contradice el enunciado, que pide sumar y luego, en caso de overflow, dejar "el contenido de los registros" — el contenido relevante de `AX` en ese momento es el resultado de la suma (el valor que causó el overflow), no un valor anterior que ya no está en ningún

registro. Además la solución oficial usa `AX` mismo como intermediario para cargar `ES` (`MOV AX, 0x800`), lo cual es lo que obliga al `PUSH / POP` en primer lugar — un enredo innecesario. La versión de arriba evita el problema por completo: nunca se toca `AX` salvo para sumar, así que en el volcado queda naturalmente la suma (el dato realmente útil para diagnosticar el overflow).

Ejercicio 2 ★★ — Copia de N bytes (memcpy)

Enunciado: dados `D1` (offset en `AX`), `D2` (offset en `BX`) y `N` (en `CX`), copiar `N` bytes consecutivos desde `D1` a `D2`. Direcciones relativas a `ES`.

Alto nivel:

```
void memcpy(byte* d1, byte* d2, short n) {
    for (short i = 0; i < n; i++)
        d2[i] = d1[i];
}
```

```
ej2 PROC
    PUSH SI
    PUSH DX                ; DX se usa como registro de trabajo (byte), se preserva

    XOR SI, SI             ; SI <- 0 (índice i)

_while:
    CMP SI, CX
    JZ _endwhile

    MOV DL, ES:[AX + SI]   ; DL <- byte en D1+i (direccionamiento indexado, EA=AX+SI)
    MOV ES:[BX + SI], DL   ; (D2+i) <- DL

    INC SI
    JMP _while

_endwhile:
    POP DX
    POP SI
    RET
ej2 ENDP
```

Revisión: `AX` (=D1), `BX` (=D2) y `CX` (=N) nunca se modifican durante el bucle (son solo leídos), por lo que quedan preservados sin necesidad de `PUSH / POP` explícito. `SI` es el único índice y se restaura al salir. `DL` se usa como registro de trabajo de 1 byte porque `N` cuenta **bytes**, no palabras.

⚠ **Error encontrado en el material oficial (el más importante de este ejercicio):** la solución de la cátedra usa `DI` (registro de **16 bits**) como intermediario: `MOV DI, ES:[AX+SI]` seguido de `MOV ES:[BX+SI], DI`. Como `DI` es una palabra completa, cada iteración del bucle copia **2 bytes** (el byte en `D1+i` y el byte en `D1+i+1`, leídos juntos como una palabra), pero el índice `SI` solo avanza de a **1** en cada vuelta (`INC SI`). El resultado es que la rutina no copia `N` bytes consecutivos sin solapamiento de lecturas: en la posición `i=1` se vuelve a leer (como mitad alta de la palabra) el byte que ya se había leído como mitad baja en `i=0`, y ese byte “extra” nunca formaba parte de lo que había que escribir ahí. Para `N` bytes, la rutina termina escribiendo datos parcialmente incorrectos en la mitad

superior del destino cada vez que “wordifica” de más al leer un byte que en la fuente real no correspondía a esa posición. La corrección es usar un registro de **8 bits** (`DL` arriba, en vez de `DI`) para que cada iteración mueva exactamente 1 byte, consistente con que `N` (en `CX`) cuenta bytes y el índice avanza de a 1.

Ejercicio 3 ★★ — Comparación de N bytes

Enunciado: comparar N bytes desde `D1` (en `SI`) con N bytes desde `D2` (en `DI`); `N` en `AX`. Devolver `Z=1` si son iguales, `Z=0` si no. Direcciones relativas a `DS`.

```
bool comparar(byte* d1, byte* d2, short n) {
    for (short i = 0; i < n; i++)
        if (d1[i] != d2[i])
            return false;    // Z=0
    return true;             // Z=1
}
```

```
ej3 PROC
    PUSH BX
    PUSH DX                ; DX se usa como byte de trabajo, se preserva

    XOR BX, BX             ; BX <- 0 (índice i)

_while:
    CMP BX, AX
    JZ _endwhile           ; recorrimos las N posiciones sin diferencias -> Z=1 (lo dejó el CMP de arriba)

    MOV DL, DS:[SI + BX]   ; DL <- byte de D1+i
    CMP DL, DS:[DI + BX]   ; ZF <- (byte de D1+i == byte de D2+i)
    JNE _endwhile         ; si difieren, salimos YA, sin tocar más flags -> Z=0

    INC BX
    JMP _while

_endwhile:
    POP DX
    POP BX
    RET
ej3 ENDP
```

Revisión, punto por punto (tolerancia cero con las banderas):

- El `CMP BX, AX` de `_while` es el que deja `Z=1` en el camino “recorrimos todo sin diferencias” — nada se ejecuta entre ese `CMP` y el `JZ`, así que el flag llega intacto.
- El `CMP DL, DS:[DI+BX]` deja el resultado de la comparación byte a byte en `ZF`. La instrucción **inmediatamente siguiente** es `JNE _endwhile` — ninguna instrucción intermedia toca los flags entre el `CMP` y el salto que los consume. Esto es exactamente lo que hay que verificar con tolerancia cero en este tipo de ejercicio.
- Se usa `DL` (no `AX / AL`) como byte de trabajo precisamente para no interferir con `AX`, que contiene `N` y se necesita intacto en cada vuelta del `_while` para el `CMP BX, AX`.

⚠ **Error grave encontrado en el material oficial:** la solución de la cátedra hace, en este orden: `CMP AL, DS:[DI+BX]` (fija `ZF` según la comparación de bytes) → `POP AX` (no toca flags, está bien) → `INC BX` → `JNE _endwhile`. El problema es que `INC` **sí modifica** `ZF` (y `SF`, `OF`; no modifica `CF`, pero eso no ayuda acá). Entre el `CMP` que calcula el resultado de la comparación de bytes y el `JNE` que se supone debe usar ese resultado, se ejecuta un `INC BX` que **pisa el flag** con un valor nuevo — `ZF` queda determinado por si `BX+1 == 0`, algo que no tiene ninguna relación con si los bytes comparados eran iguales o no. En la práctica, esto rompe la rutina: como `BX` casi nunca vale `0xFFFF` (para que `BX+1` dé `0`), `ZF` queda casi siempre en `0` tras el `INC`, entonces `JNE` casi siempre salta a `_endwhile` — o, según el orden exacto de instrucciones en variantes similares del mismo error, la rutina puede terminar ignorando por completo las diferencias reales entre bytes y solo reportando lo que dice el último `CMP BX,AX` del tope del bucle (es decir, “llegó al final” en vez de “los bytes coincidían”). Es el clásico error de **usar una instrucción que modifica flags entre el `CMP` que importa y el salto condicional que lo consume** — el mismo tipo de error que la guía temática de este módulo señala como el más común en parciales de este tema. La corrección (arriba) mueve el `JNE` a inmediatamente después del `CMP`, y deja el `INC BX` para después (donde ya no importa si pisa flags, porque el próximo flag relevante lo vuelve a fijar el `CMP BX,AX` de la siguiente vuelta del `_while`).

Ejercicio 4 ★★ — Búsqueda en tabla de bytes

Enunciado: buscar `AL` en una tabla `[BX, DI]` (offsets en `ES`); devolver la posición relativa en `CX`, o `-1` si no está.

```
short buscar(byte elemento, byte* comienzo, byte* fin) {
    short pos = 0;
    for (byte* p = comienzo; p != fin; p++, pos++)
        if (*p == elemento) return pos;
    return -1;
}
```

```

ej4 PROC
    PUSH BX

    XOR CX, CX          ; CX <- 0 (posición actual)

_while:
    CMP BX, DI
    JE _noEncontre      ; llegamos al final de la tabla ([BX]==DI, exclusivo) sin encontrarlo

    CMP ES:[BX], AL
    JE _encontre        ; coincide: CX ya tiene la posición correcta, salimos YA (sin tocar flags de
por medio)

    INC CX
    INC BX
    JMP _while

_noEncontre:
    MOV CX, -1

_encontre:
    POP BX
    RET
ej4 ENDP

```

Esta parte del material oficial (ver PDF) **es correcta**: el `CMP ES:[BX],AL` está seguido inmediatamente por el `JE` que lo consume, sin instrucciones intermedias que pisen `ZF` — a diferencia del Ejercicio 3, acá no hay bug de flags. Único punto a documentar (no es un error, es una decisión de diseño que la letra deja abierta): `DI` se interpreta como límite **exclusivo** (una posición más allá del último byte válido de la tabla, al estilo `end()` de un iterador), de forma que `BX==DI` señala “recorrimos toda la tabla” sin necesidad de comparar también el elemento en `DI`.

Ejercicio 5 ★★ — Suma de vector con detección de posición del carry

Enunciado: sumar 1024 palabras desde `VECTOR` (apuntado por `ES:BX`) en orden ascendente; retornar en `AX` la **posición** del sumando en que ocurre `CARRY`, o `0` si no hay acarreo.

```

short sumarConCarry(word* vector) {
    short suma = 0;
    for (short i = 0; i < 1024; i++) {
        suma = suma + vector[i];    // con acarreo -> CF
        if (hubo acarreo) return i; // devolver la POSICIÓN (índice), no el valor
    }
    return 0;
}

```



```

ej5 PROC
    PUSH SI

    XOR AX, AX          ; AX <- 0 (acumulador de la suma)
    XOR SI, SI          ; SI <- 0 (offset en bytes: 0,2,4,...)

_while:
    CMP SI, 2048        ; 1024 palabras * 2 bytes = 2048 (verificado: 1024*2=2048)
    JZ  _noHayCarry
    ADD AX, ES:[BX + SI]
    JB  _hayCarry        ; JB = CF=1 (acarreo SIN SIGNO, exactamente lo que pide "CARRY")
    ADD SI, 2
    JMP _while

_hayCarry:
    MOV AX, SI
    SHR AX, 1           ; posición = offset en bytes / 2 (índice de palabra, 0-based)
    JMP _return

_noHayCarry:
    XOR AX, AX

_return:
    POP SI
    RET
ej5 ENDP

```

Revisión: `JB` (Jump if Below, equivalente a `JC`) es el salto correcto para probar `CF` — la letra pide explícitamente “CARRY”, que es sin signo (`CF`), no “overflow” con signo (`OF`); usar `JO` acá sería el error típico de confundir ambas banderas (ver guía temática §2.4). En el momento del `JB`, `SI` todavía apunta al offset en bytes del elemento que se acaba de sumar (el `ADD SI, 2` de avance recién se ejecuta si NO hubo carry), así que `SI/2` da directamente el índice (posición 0-based) de ese elemento.

⚠ Error encontrado en el material oficial: la solución de la cátedra, en la etiqueta `_hayCarry`, hace `MOV AX, ES:[BX+SI]` — esto vuelve a leer de memoria y deja en `AX` el **valor** del elemento del vector en esa posición, no su **posición**. La letra pide explícitamente: “*retorne en AX la **posición** del sumando en el cual ocurre CARRY*” — posición e índice, no el dato. La corrección (arriba) usa `SI` (que en ese punto todavía contiene el offset en bytes del elemento que disparó el acarreo) para calcular la posición como índice de palabra (`SI` dividido 2, con `SHR AX, 1`), en vez de leer el contenido del vector.

Ejercicio 6 ★★ — Eliminar espacios de un string

Enunciado: eliminar los caracteres `' '` (0x20) de un string en `ES:BX`, dejando el resultado en `ES:DI`. Sin solapamiento entre ambos strings. Fin de string marcado por `NULL` (0x00).

(Este ejercicio no tiene solución en el material oficial — se resuelve íntegramente acá.)

```

void quitarEspacios(char* origen, char* destino) {
    short j = 0;
    for (short i = 0; origen[i] != '\0'; i++) {
        if (origen[i] != ' ')
            destino[j++] = origen[i];
    }
    destino[j] = '\0';
}

```

```

ej6 PROC
    PUSH AX
    PUSH BX
    PUSH DI

    _bucle:
        MOV AL, ES:[BX]          ; caracter actual del string origen
        CMP AL, 0
        JE _fin                 ; NULL: terminamos de recorrer el origen

        CMP AL, 0x20             ; ¿es un espacio?
        JE _saltar

        MOV ES:[DI], AL          ; copiamos el caracter al destino
        INC DI

    _saltar:
        INC BX
        JMP _bucle

    _fin:
        MOV BYTE PTR ES:[DI], 0 ; terminamos el string destino con NULL (BYTE PTR: tamaño ambiguo sin él)
        POP DI
        POP BX
        POP AX
        RET
ej6 ENDP

```

Revisión: `BX` y `DI` se usan directamente como punteros móviles (se incrementan in-place) en vez de mantener índices separados — es válido porque la letra no exige preservar los valores de entrada de `BX` / `DI` con algún significado posterior, pero de todos modos se los restaura con `PUSH` / `POP` al final por prolijidad (mismo estilo que el resto del práctico). `BYTE PTR` es obligatorio en `MOV ES:[DI], 0` porque `[DI]` solo, sin otro operando que fije el tamaño, es ambiguo para el ensamblador (podría interpretarse como escribir 1 byte o 2) — exactamente el caso que señala la guía temática §5 sobre `byte ptr` / `word ptr`.

Ejercicio 7 ★ — Layout de struct en memoria

Enunciado: distribución en memoria y tamaño de `mi_var[3]`, donde `mi_tipo` es `{ char letras[4]; short enteros[2]; char letras_MAYUS[2]; }`.

```
char[4]  -> 4 bytes x 1 byte/char = 4 bytes
short[2] -> 2 bytes/short x 2      = 4 bytes
char[2]  -> 1 byte/char x 2       = 2 bytes
```

```
-----
mi_tipo  -> 4 + 4 + 2 = 10 bytes por elemento
mi_var[3] -> 10 bytes x 3 = 30 bytes en total
```

(Verificado por cálculo independiente: $(4+4+2)*3 = 30$. ✓)

Distribución de `mi_var[0]` (los otros dos elementos se repiten idénticos a partir del offset 10 y 20 respectivamente, sin padding — el 8086 no impone alineación obligatoria y el curso asume layout empaquetado):

Offset relativo	0	1	2	3	4–5	6–7	8	9
Campo	letras[0]	letras[1]	letras[2]	letras[3]	enteros[0]	enteros[1]	letras_MAYUS[0]	letras_MAYUS[1]
Tamaño	1B	1B	1B	1B	2B	2B	1B	1B

`mi_var[1]` ocupa los offsets 10–19, `mi_var[2]` los offsets 20–29 (cada elemento repite exactamente esta misma distribución de 10 bytes).

Ejercicio 8 ★★ — Substring (buscar un string dentro de un texto)

Enunciado: `short substr(char* texto, char* buscado)`: posición de la primera ocurrencia de `buscado` en `texto`, o `-1`. `DS:SI` apunta a `texto`, `DS:DI` a `buscado`, resultado en `AX`.

(Ejercicio sin solución en el material oficial.)

(a) Alto nivel:

```
short substr(char* texto, char* buscado) {
    short i = 0;
    while (texto[i] != '\0') {
        short j = 0;
        while (buscado[j] != '\0' && texto[i + j] == buscado[j])
            j++;
        if (buscado[j] == '\0')
            return i;          // "buscado" se agotó sin fallar -> coincidencia completa
        i++;
    }
    return -1;
}
```

(b) Compilación a 8086:

Solo 4 registros pueden aparecer dentro de corchetes en 8086: `BX`, `BP`, `SI`, `DI`. Necesitamos direccionar simultáneamente 4 cosas (inicio de `texto`, inicio de `buscado`, posición `i`, posición `j`) — exactamente 4 registros disponibles, pero las combinaciones válidas de base+índice (`BX+SI`, `BX+DI`, `BP+SI`, `BP+DI`) no alcanzan

para sumar tres términos (`texto+i+j`) en una sola instrucción. La solución estándar es no mantener `i` y `j` como índices separados, sino usar **punteros que avanzan** directamente:

- `SI` : puntero al intento actual (arranca en `texto`, avanza de a 1 en el bucle externo — es literalmente `texto+i`).
- `DI` : puntero fijo al inicio de `buscado` (nunca se modifica — se necesita para “reiniciar” el bucle interno en cada intento).
- `BX` : copia de `SI` al entrar a cada intento — puntero de trabajo que recorre `texto` en el bucle interno.
- `BP` : copia de `DI` al entrar a cada intento — puntero de trabajo que recorre `buscado` en el bucle interno. **Ojo:** `BP` usa `SS` por defecto (no `DS`), así que hay que forzar el segmento explícitamente con `DS:` en cada acceso vía `BP` .
- `CX` : guarda el valor **original** de `SI` (el inicio real de `texto`), para poder calcular `i = SI_actual - texto_original` al encontrar una coincidencia.

```

; Entrada: DS:SI = texto, DS:DI = buscado
; Salida: AX = posición de la primera ocurrencia, o -1 si no se encuentra
substr PROC
    PUSH BX
    PUSH CX
    PUSH DX
    PUSH BP
    PUSH SI
    PUSH DI

    MOV CX, SI                ; CX <- texto_base (offset original de "texto", para el cálculo final)

_outer:
    MOV AL, [SI]              ; caracter actual de texto (DS por defecto vía SI)
    CMP AL, 0
    JE _noEncontrado          ; fin de "texto" sin encontrar coincidencia completa

    MOV BX, SI                ; puntero de trabajo sobre texto, arranca en el intento actual (texto+i)
    MOV BP, DI                ; puntero de trabajo sobre buscado, reinicia en su comienzo

_inner:
    MOV DL, DS:[BP]           ; caracter actual de "buscado" (override DS: BP por defecto usa SS)
    CMP DL, 0
    JE _matchCompleto         ; "buscado" se agotó sin fallar -> coincidencia completa

    MOV AL, [BX]              ; caracter actual de "texto" en esta posición del intento
    CMP AL, DL
    JNE _siguienteIntento     ; difieren: abortamos este intento, probamos el próximo i

    INC BX
    INC BP
    JMP _inner

_matchCompleto:
    MOV AX, SI
    SUB AX, CX                ; AX <- (texto+i) - texto_base = i
    JMP _fin

_siguienteIntento:
    INC SI                    ; probamos el próximo carácter de inicio (i++)
    JMP _outer

_noEncontrado:
    MOV AX, -1

_fin:
    POP DI
    POP SI
    POP BP
    POP DX
    POP CX
    POP BX
    RET
substr ENDP

```

Revisión: en `_inner`, el `CMP AL,DL` está seguido inmediatamente por el `JNE` que lo consume — sin instrucciones intermedias, evitando el error de flags del Ejercicio 3. El chequeo de fin de `buscado` (`CMP DL,0` / `JE _matchCompleto`) se hace **antes** de comparar los caracteres de `texto`, igual que el cortocircuito `&&` del

pseudocódigo en alto nivel (si `buscado[j]` ya es `NULL`, no hace falta mirar `texto[i+j]`).

(c) ¿Se puede acelerar conociendo los largos? Sí, de varias formas:

1. **Evitar comparar contra `NULL` en el bucle interno:** si se conoce `m = largo(buscado)`, se puede usar un contador que arranca en `m` y se decrementa (`DEC` + `JNZ`, o directamente `LOOP`) en vez de comparar cada carácter contra `0` — ahorra una comparación por iteración.
2. **Corte temprano del bucle externo:** si se conoce `n = largo(texto)`, en cuanto la posición restante `n - i` sea menor que `m`, ya no puede haber una coincidencia completa y se puede terminar sin seguir recorriendo `texto` hasta su `NULL` — en el peor caso (patrón que no aparece) esto evita hasta `m` comparaciones inútiles por cada posición cerca del final.
3. **Usar las instrucciones de string nativas del 8086** (`REPE CMPSB`): con los largos conocidos, se puede cargar `CX = m` y usar `REPE CMPSB` para comparar hasta `m` bytes con auto-incremento de `SI` / `DI` en una sola instrucción repetida por hardware, en vez de un bucle manual — más compacto y, en la práctica, más rápido que el bucle instrucción por instrucción de arriba.

Ejercicio 9 ★★ — Peso de un arreglo de matrices 4×4

Enunciado: recorrer un arreglo de `CANT_MATRICES` matrices 4×4 de enteros con signo (offset `MATRICES`, en `ES`), sumar los elementos de cada una y guardar los resultados en `pesosMatrices` (offset `PESOS_MATRICES`, en `DS`).

(Ejercicio sin solución en el material oficial.)

```
for (short i = 0; i < CANT_MATRICES; i++) {
    short suma = 0;
    for (short k = 0; k < 16; k++)          // 4x4 = 16 elementos por matriz
        suma += matrices[i].elemento[k];  // consecutivos en memoria, por filas
    pesosMatrices[i] = suma;
}
```

```

sumaMatrices PROC
    PUSH AX
    PUSH BX
    PUSH CX
    PUSH DX
    PUSH SI
    PUSH DI

    MOV BX, MATRICES      ; BX <- comienzo del arreglo de matrices (offset en ES)
    MOV SI, PESOS_MATRICES ; SI <- comienzo del arreglo de pesos (offset en DS)
    MOV CX, CANT_MATRICES  ; CX <- cantidad de matrices a procesar

_forMatriz:
    CMP CX, 0
    JZ _finForMatriz

    XOR AX, AX            ; AX <- 0 (acumulador de la suma de ESTA matriz)
    MOV DI, BX            ; DI <- comienzo de la matriz actual
    MOV DX, 16            ; DX <- 16 elementos (4x4) a sumar

_forElemento:
    CMP DX, 0
    JZ _finForElemento
    ADD AX, ES:[DI]        ; sumamos el elemento actual (short con signo)
    ADD DI, 2              ; siguiente elemento (2 bytes por short)
    DEC DX
    JMP _forElemento

_finForElemento:
    MOV [SI], AX          ; pesosMatrices[i] <- suma (DS por defecto)
    ADD SI, 2              ; siguiente casillero de pesosMatrices
    ADD BX, 32             ; siguiente matriz (4*4*2 = 32 bytes, verificado)
    DEC CX
    JMP _forMatriz

_finForMatriz:
    POP DI
    POP SI
    POP DX
    POP CX
    POP BX
    POP AX
    RET
sumaMatrices ENDP

```

Revisión: todas las comparaciones de control de bucle (`CMP CX,0` , `CMP DX,0`) son explícitas y están inmediatamente seguidas del `JZ` correspondiente — se evita deliberadamente apoyarse en los flags que deja `DEC` (que sí los modifica, pero acá no se usa ese flag para nada porque el `CMP` explícito ya lo refija antes de cada chequeo), el mismo cuidado que corrigió el bug del Ejercicio 3. Los accesos a la matriz (`ES:[DI]`) fuerzan el segmento `ES` porque así lo indica la letra (“MATRICES: desplazamiento dentro de ES”); el acceso a `pesosMatrices` usa el `DS` por defecto sin necesidad de override, porque `PESOS_MATRICES` está declarado en `DS`.

Ejercicio 10 ★★ — Protocolo de pasaje de parámetros por pila

Enunciado: protocolo: los argumentos se apilan en el orden en que se declaran; la función devuelve el resultado en `AX` preservando los demás registros. **(a)** compilar `AND(x, y, z: short) return short`. **(b)** programa que invoque a esa función para calcular `AX AND BX AND CX`.

⚠ **Nota sobre el protocolo:** la letra no especifica quién limpia la pila después del `RET` (si el llamador o la propia función con `RET n`). Se adopta acá la misma convención que usa la guía temática de este módulo para el ejemplo de `factorial` (sección 6): **el llamador es responsable de limpiar los parámetros con `ADD SP, n` después del `RET`**, por consistencia con el resto del curso. También hay que tener cuidado con el **nombre**: `AND` es una instrucción real del 8086, no se puede usar como nombre de `PROC` en un ensamblador real (MASM/TASM lo tomaría como la mnemónica) — se usa `AND_` como nombre de la rutina.

(a) Con `PUSH x; PUSH y; PUSH z; CALL AND_` (orden de declaración, `x` primero), y como la pila crece hacia abajo, el último apilado (`z`) queda más cerca del tope: dentro de la función, con el frame armado, `[BP+4]=z`, `[BP+6]=y`, `[BP+8]=x` (BP viejo en `[BP+0]`, dirección de retorno *near* de 2 bytes en `[BP+2]`).

```
; function AND(x, y, z: short) return short
; Protocolo: PUSH x; PUSH y; PUSH z; CALL AND_ (orden declarado, x primero)
; Resultado en AX. El llamador limpia con ADD SP,6 tras el RET.

AND_ PROC
    PUSH BP
    MOV BP, SP          ; [BP+4]=z, [BP+6]=y, [BP+8]=x
    PUSH BX              ; preservamos BX, usado como registro de trabajo

    MOV AX, [BP+8]       ; AX <- x
    MOV BX, [BP+6]       ; BX <- y
    AND AX, BX           ; AX <- x AND y
    MOV BX, [BP+4]       ; BX <- z
    AND AX, BX           ; AX <- (x AND y) AND z

    POP BX               ; restauramos BX
    POP BP               ; restauramos BP del llamador
    RET                  ; el llamador debe hacer ADD SP,6
AND_ ENDP
```

(b) Programa que invoca a `AND_` para calcular `AX AND BX AND CX`, dejando el resultado en `AX`:

```
PUSH AX      ; primer argumento (x): valor original de AX
PUSH BX      ; segundo argumento (y): valor original de BX
PUSH CX      ; tercer argumento (z): valor original de CX
CALL AND_
ADD SP, 6    ; el llamador retira los 3 parámetros (3 x 2 bytes)
; resultado (AX and BX and CX) queda en AX; BX y CX no se modificaron
```

Revisión: como `AX`, `BX` y `CX` se copian a la pila **por valor** antes del `CALL`, no importa que `AND_` sobrescriba `AX` con el resultado — los valores originales ya están a salvo en la pila en el momento de la llamada. `AND_` preserva `BX` internamente (push/pop), cumpliendo el requisito del protocolo de “mantener los valores de los demás registros”.

Resumen de técnicas usadas en este práctico

Ejercicio	Técnica
1	Detección de overflow con signo vía <code>OF</code> / <code>JNO</code> ; carga de un registro de segmento vía un registro general intermediario
2, 6	Recorrido byte a byte con registro de 8 bits como temporal (evitar copiar palabras cuando se pide copiar bytes)
3, 8, 9	Comparación con salto condicional inmediatamente después del <code>CMP</code> que lo genera — ninguna instrucción que modifique flags en el medio
4	Búsqueda lineal con límite exclusivo (<code>[inicio, fin)</code>)
5	Distinción <code>CF</code> (CARRY, sin signo) vs <code>OF</code> (overflow con signo); cálculo de posición vía aritmética de punteros/offsets, no lectura de datos
7	Layout de struct: suma de tamaños de campos (sin padding en este curso), replicado para arreglos de structs
8	Direccionamiento con solo 4 registros habilitados para memoria (<code>BX, BP, SI, DI</code>); simulación de “3 índices” mediante punteros que avanzan en vez de sumar offsets en una sola instrucción; override explícito de segmento (<code>DS:[BP]</code>) cuando se usa <code>BP</code> fuera de la pila
9	Doble bucle anidado con puntero base + puntero de recorrido interno, avance de puntero base en bloques de tamaño fijo (32 bytes por matriz)
10	Protocolo de pasaje de parámetros por pila con frame <code>BP</code> ; cálculo de offsets <code>[BP+4]</code> , <code>[BP+6]</code> , <code>[BP+8]</code> según orden de apilado; limpieza de pila a cargo del llamador

Para el examen: el error más peligroso de este práctico (Ejercicio 3) es el patrón “instrucción que modifica flags interpuesta entre el `CMP` que importa y el salto condicional que lo consume” — cualquier instrucción aritmética (`ADD`, `SUB`, `INC`, `DEC`) o lógica (`AND`, `OR`, `XOR`) entre medio destruye el flag que se necesitaba. Regla de tolerancia cero: después de un `CMP` cuyo resultado se va a usar, el **próximo** salto condicional debe ser el que lo consuma — si hace falta hacer algo más antes, hacerlo **después** del salto, no antes. El segundo patrón recurrente es la confusión `CF` vs `OF` (acarreo sin signo vs overflow con signo, Ejercicios 1 y 5) — siempre verificar qué palabra usa el enunciado (“overflow” → con signo → `OF`; “carry”/“acarreo” → sin signo → `CF`). El tercero es recordar que solo `BX`, `BP`, `SI`, `DI` pueden aparecer dentro de corchetes en una dirección de memoria (Ejercicio 8); si el problema necesita más de dos punteros/índices simultáneos, hay que usar punteros que avanzan en el bucle en vez de intentar sumar tres registros en una sola instrucción de direccionamiento.